Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory work work #3.2

Signal Acquisition and Advanced Conditioning.

During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur Cravtov
std. gr. FAF-231

Verified by:
Alexei Martîniuc
asist. univ

Chișinău, 2026

Contents

1 Objective and Technical Analysis **1**
1.1 Signal Conditioning Techniques 2
1.1.1 Saturation (Clamping) 2
1.1.2 Median Filter (Salt-and-Pepper Noise) 2
1.1.3 Exponential Moving Average (EMA) 2

2 System Design **2**
2.1 System Architecture 2
2.2 Conditioning Pipeline Flow 3

3 Implementation and Results **3**
3.1 Software Architecture 3
3.2 Layered Design 4
3.2.1 Distance Sensor Driver 4
3.2.2 Temperature Sensor Driver 5
3.3 Reporting and Visualization 5

4 Conclusion **6**

References **6**

A Source Code **7**

1 Objective and Technical Analysis

The objective of this laboratory work is to implement an advanced signal acquisition and conditioning pipeline for embedded systems. This involves not only reading raw data from sensors but also applying mathematical filters to remove noise and ensure data integrity. The system implements a three-stage conditioning pipeline: **Saturation**, **Median Filtering** (for salt-and-pepper noise), and **Exponential Moving Average (EMA)** for smoothing.

Through this work, the following learning outcomes are pursued:

- Implementing a multi-stage signal conditioning pipeline.
- Understanding and implementing non-linear filters (Median) and recursive linear filters (EMA).
- Managing intermediate data states (raw, saturated, median, filtered) in a multi-tasking environment.

- Real-time visualization of the conditioning process via serial interface and ASCII graphs.

1.1 Signal Conditioning Techniques

1.1.1 Saturation (Clamping)

Saturation is the simplest form of conditioning, ensuring that the input signal remains within a predefined physical or logical range $[min, max]$. It prevents outliers or sensor malfunctions from propagating "impossible" values through the system.

$$y[n] = \begin{cases} max & \text{if } x[n] > max \\ min & \text{if } x[n] < min \\ x[n] & \text{otherwise} \end{cases} \tag{1}$$

1.1.2 Median Filter (Salt-and-Pepper Noise)

The Median filter is a non-linear digital filtering technique, often used to remove noise while preserving edges. It is particularly effective against "salt-and-pepper" noise (random isolated spikes). For a window of size N, the filter outputs the middle value of the sorted window samples.

$$y[n] = \text{median}\{x[n], x[n-1], \ldots, x[n-N+1]\} \tag{2}$$

1.1.3 Exponential Moving Average (EMA)

EMA is a type of infinite impulse response (IIR) filter that applies weighting factors which decrease exponentially. It is used to smooth the signal and reduce high-frequency noise. The smoothing factor α (between 0 and 1) determines the balance between noise reduction and responsiveness.

$$y[n] = \alpha \cdot x[n] + (1-\alpha) \cdot y[n-1] \tag{3}$$

2 System Design

2.1 System Architecture

The system architecture follows a modular task-based design using FreeRTOS.

- **Acquisition Task**: Reads raw data from sensors (NTC Thermistor and Ultrasonic Sensor).
- **Conditioning Task**: Consumes raw values from a queue and applies the `SignalConditioner` pipeline.

- **Report Task**: Periodic reporting of the entire pipeline state with color-coded console output and ASCII graphs.

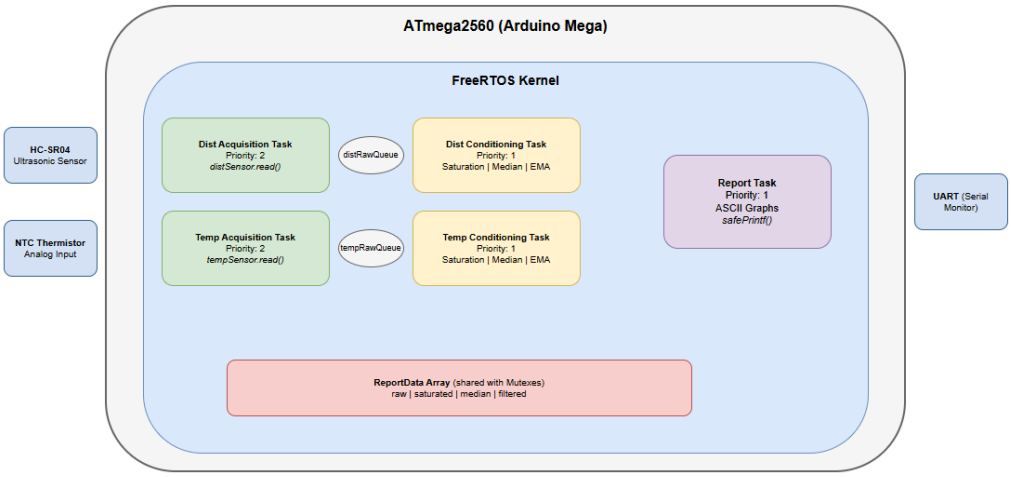

Figure 1: System Architecture Diagram for Lab 3.2

2.2 Conditioning Pipeline Flow

Figure 2 illustrates the data flow through the `SignalConditioner` class. Each sample is first clamped, then stored in a circular buffer for median calculation, and finally passed through the EMA stage.

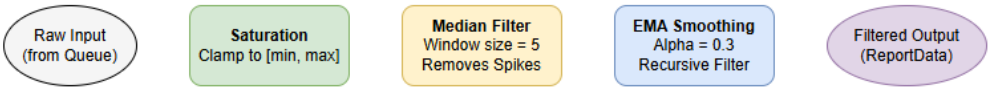

Figure 2: Signal Conditioning Pipeline Flowchart

3 Implementation and Results

3.1 Software Architecture

The `SignalConditioner` class encapsulates the mathematical logic, while the `ConditioningTask` manages the FreeRTOS integration.

- **Wait for Data**: Task blocks on `xQueueReceive` until a raw sample arrives.

- **Process**: Calls `conditioner.addSample(raw)`.
- **Shared State**: Updates the `ReportData` structure protected by a Mutex, storing all intermediate values (`raw`, `saturated`, `median`, `filtered`).

3.2 Layered Design

3.2.1 Distance Sensor Driver

Figure 3 shows the layer decomposition for the pulse-based digital sensor.

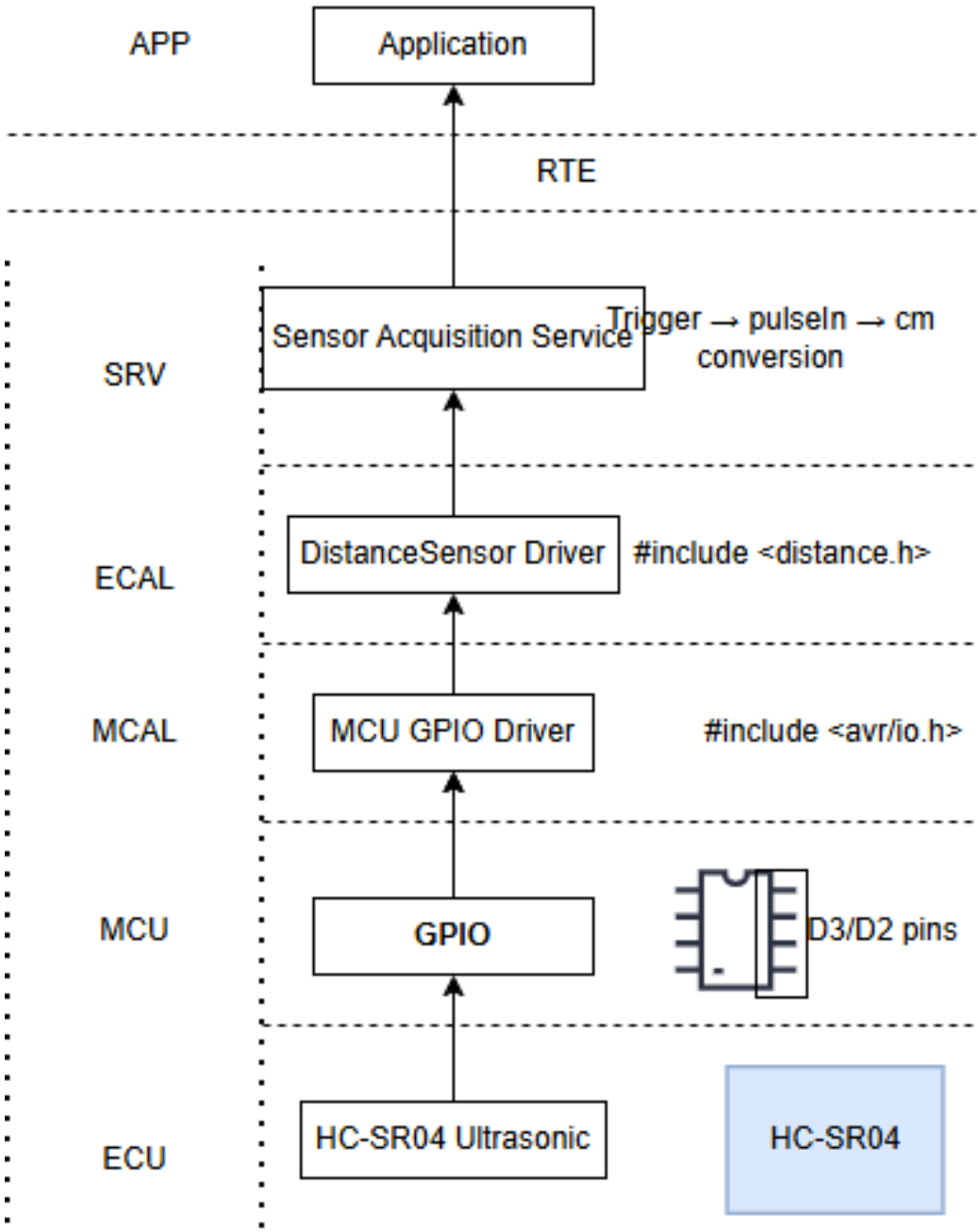

Figure 3: Distance Sensor layering

3.2.2 Temperature Sensor Driver

Figure 4 shows the layer decomposition for the analog sensor. The ECAL layer here includes the mathematical transformation from ADC counts to Celsius.

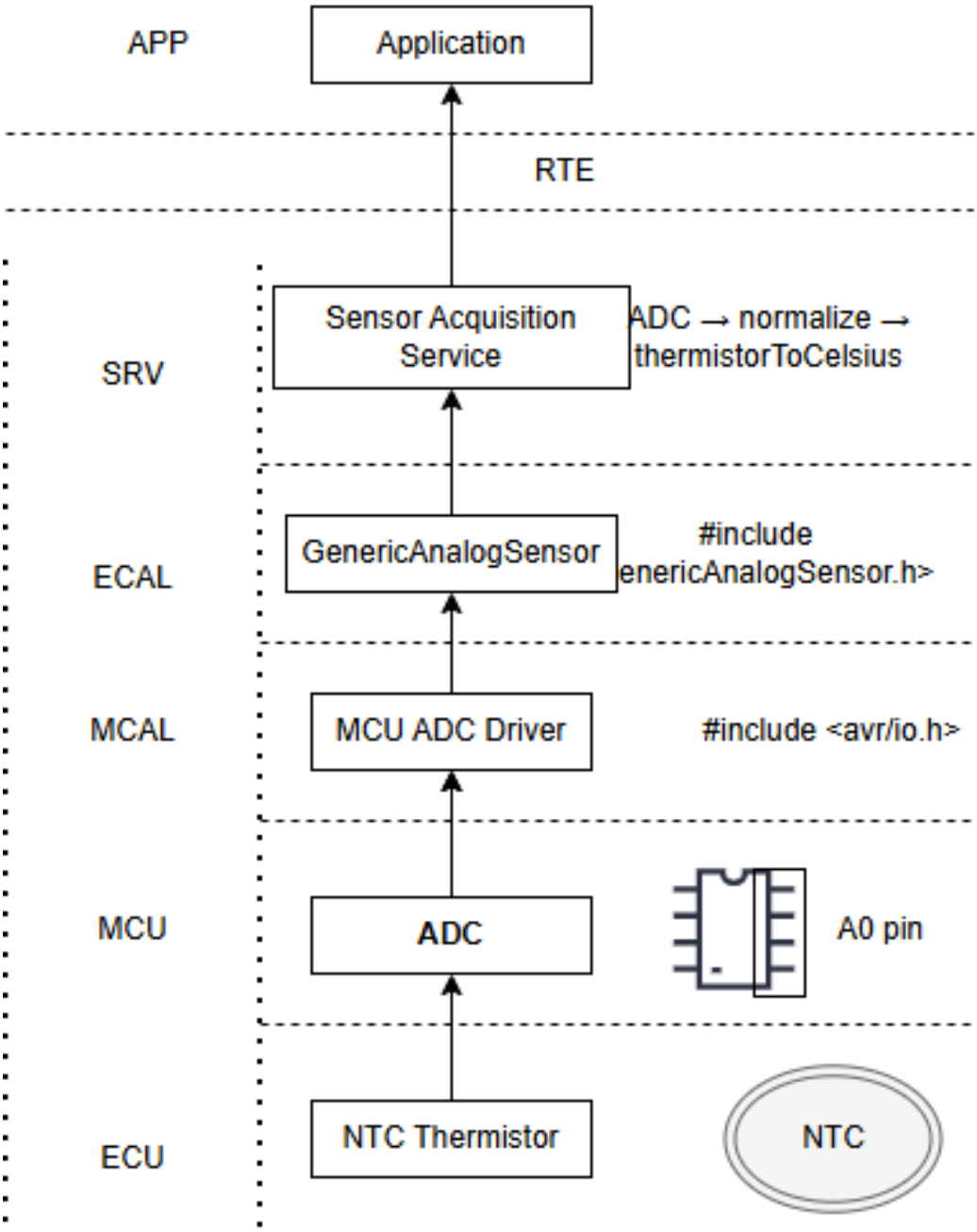

Figure 4: Temperature Sensor layering

3.3 Reporting and Visualization

A key feature of this implementation is the enhanced reporting. The `ReportTask` outputs a detailed breakdown of the pipeline for each sensor:

- **Status**: OK/LOW/HIGH based on saturation limits.

- **Pipeline Path**: Visual string showing $Raw \rightarrow Sat \rightarrow Med \rightarrow Filtered$.
- **ASCII Graph**: A dynamic bar representing the filtered value within its configured range.

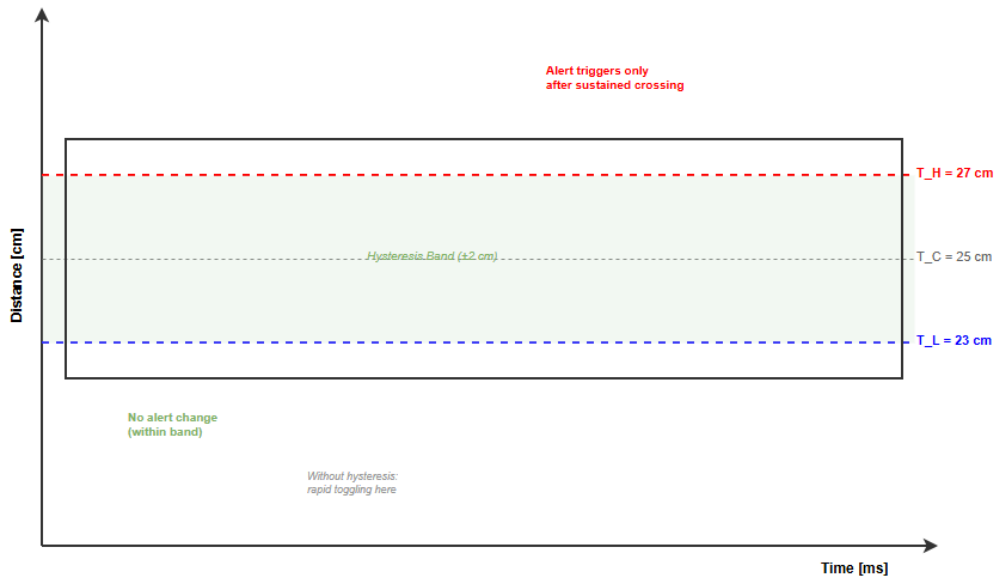

Figure 5: Console Visualization: Real-time Pipeline and ASCII Graph

4 Conclusion

This laboratory work successfully implemented a robust signal conditioning system. By combining saturation, median filtering, and EMA, we created a pipeline that is resilient to both isolated noise spikes and continuous high-frequency fluctuations. The modular architecture using FreeRTOS ensures that conditioning can run independently of acquisition, while the detailed reporting provides full visibility into the system's internal state.

References

[1] "Median Filter Performance in Salt-and-Pepper Noise Removal", Signal Processing Journal.

[2] "Digital Filtering with Exponential Moving Average", Embedded Systems Design.

[3] FreeRTOS API Reference, `https://www.freertos.org/a00106.html`

A Source Code

Full source code for Lab 3.2 is available in the repository.

`SignalConditioner.cpp`

```cpp
#include "SignalConditioner.h"

SignalConditioner::SignalConditioner(float minSat, float maxSat, uint8_t windowSize, float emaAlpha)
    : minSat(minSat), maxSat(maxSat), bufSize(windowSize),
      index(0), count(0), emaAlpha(emaAlpha), emaValue(0.0f), emaInitialised(false),
      lastRaw(0.0f), lastSaturated(0.0f), lastMedian(0.0f)
{
    buffer = new float[bufSize];
    for (uint8_t i = 0; i < bufSize; i++) buffer[i] = 0.0f;
}

float SignalConditioner::saturate(float value) const {
    if (value < minSat) return minSat;
    if (value > maxSat) return maxSat;
    return value;
}

float SignalConditioner::computeMedian() const {
    // Copy to temp array, then sort (bubble sort tiny N)
    uint8_t n = count < bufSize ? count : bufSize;
    float tmp[7]; // max window we ever use
    for (uint8_t i = 0; i < n; i++) tmp[i] = buffer[i];

    for (uint8_t i = 0; i < n - 1; i++) {
        for (uint8_t j = 0; j < n - i - 1; j++) {
            if (tmp[j] > tmp[j + 1]) {
                float t    = tmp[j];
                tmp[j]     = tmp[j + 1];
                tmp[j + 1] = t;
            }
        }
    }
    return tmp[n / 2];
```

```
}

void SignalConditioner::addSample(float raw) {
    lastRaw = raw;
    float sat = saturate(raw);
    lastSaturated = sat;
    buffer[index] = sat;
    index = (index + 1) % bufSize;
    if (count < bufSize) count++;

    // Compute median then apply EMA
    float median = computeMedian();
    lastMedian = median;
    if (!emaInitialised) {
        emaValue = median;
        emaInitialised = true;
    } else {
        emaValue = emaAlpha * median + (1.0f - emaAlpha) *
            emaValue;
    }
}

float SignalConditioner::getFiltered() const {
    return emaValue;
}

bool SignalConditioner::isReady() const {
    return count >= bufSize;
}

float SignalConditioner::getLastRaw() const        { return
    lastRaw; }
float SignalConditioner::getLastSaturated() const  { return
    lastSaturated; }
float SignalConditioner::getLastMedian() const     { return
    lastMedian; }
float SignalConditioner::getMin() const            { return
    minSat; }
```

```
float SignalConditioner::getMax() const { return maxSat; }
```

Listing 1: SignalConditioner.cpp

`ConditioningTask.cpp`

```
#include "ConditioningTask.h"
#include <semphr.h>

ConditioningTask::ConditioningTask(SignalConditioner* conditioner)
    : conditioner(conditioner), inQueue(NULL), reportEntry(NULL) {}

void ConditioningTask::setInQueue(QueueHandle_t q)      { inQueue = q; }
void ConditioningTask::setReportEntry(ReportData* r)    { reportEntry = r; }

void ConditioningTask::run() {
    while (true) {
        float raw = 0.0f;
        if (inQueue != NULL && xQueueReceive(inQueue, &raw, portMAX_DELAY) == pdTRUE) {
            conditioner->addSample(raw);

            // Update the shared report entry with the conditioned values
            if (reportEntry != NULL && reportEntry->xSemaphore != NULL) {
                xSemaphoreTake(reportEntry->xSemaphore, portMAX_DELAY);
                reportEntry->raw        = conditioner->getLastRaw();
                reportEntry->saturated  = conditioner->getLastSaturated();
                reportEntry->median     = conditioner->getLastMedian();
                reportEntry->filtered   = conditioner->getFiltered();
```

```

                // Simple status check check against min/max saturation
                if (reportEntry->raw < conditioner->getMin()) reportEntry->status = -1;
                else if (reportEntry->raw > conditioner->getMax()) reportEntry->status = 1;
                else reportEntry->status = 0;

                xSemaphoreGive(reportEntry->xSemaphore);
            }
        }
    }
}
```

Listing 2: ConditioningTask.cpp

`ReportTask.cpp`

```
#include "ReportTask.h"
#include "SafePrintf.h"
#include <stdlib.h>
#include <Arduino.h>

ReportTask::ReportTask(ReportData* data, uint8_t numberOfSensors, uint32_t delay)
    : reportData(data), numberOfSensors(numberOfSensors), delay(delay) {}

void ReportTask::run() {
    static char fRaw[10], fSat[10], fMed[10], fFil[10];

    while (true) {
        // —— Human-readable & Colorful Pipeline Report ——
        for (uint8_t i = 0; i < numberOfSensors; i++) {
            if (reportData[i].xSemaphore != NULL) {
                xSemaphoreTake(reportData[i].xSemaphore, portMAX_DELAY);

                int8_t s = reportData[i].status;
                float r = reportData[i].raw;
```

```
                float sa = reportData[i].saturated;
                float m = reportData[i].median;
                float f = reportData[i].filtered;
                float mn = reportData[i].min;
                float mx = reportData[i].max;

                xSemaphoreGive(reportData[i].xSemaphore);

                dtostrf(r, 6, 2, fRaw);
                dtostrf(sa, 6, 2, fSat);
                dtostrf(m, 6, 2, fMed);
                dtostrf(f, 6, 2, fFil);

                // —— Fancy Console Output ——
                // \x1b[32m - Green, \x1b[31m - Red, \x1b[33m - Yellow, \x1b[36m - Cyan, \x1b[0m - Reset

                const char* statusColor = "\x1b[32m"; // Green OK
                const char* statusStr = "OK";
                const char* barColor = "\x1b[32m";

                if (s < 0) {
                    statusColor = "\x1b[33m"; // Yellow
                    statusStr = "LOW";
                    barColor = "\x1b[33m";
                } else if (s > 0) {
                    statusColor = "\x1b[31m"; // Red
                    statusStr = "HIGH";
                    barColor = "\x1b[31m";
                }

                safePrintf("\x1b[1m[%s]\x1b[0m Status: %s%s\x1b[0m | Value: \x1b[36m%s\x1b[0m\n",
                    reportData[i].sensorName, statusColor, statusStr, fFil);

                safePrintf("  Pipeline: Raw:%s -> Sat:%s -> Med
```

```
                :%s -> Filtered:%s\n",
                fRaw, fSat, fMed, fFil);

            // -- ASCII Graph --
            int width = 20;
            float range = mx - mn;
            float percentage = (range > 0.001f) ? (f - mn) / range : 0.0f;
            if (percentage < 0.0f) percentage = 0.0f;
            if (percentage > 1.0f) percentage = 1.0f;
            int pos = (int)(percentage * width);

            safePrintf("  Graph: [");
            for (int j = 0; j < width; j++) {
                if (j < pos) safePrintf("%s#\x1b[0m", barColor);
                else if (j == pos) safePrintf("\x1b[33m|\x1b[0m");
                else safePrintf("-");
            }
            safePrintf("] %d%%\n", (int)(percentage * 100));
        }
    }

    // -- Teleplot output (>label:value format) --
    for (uint8_t i = 0; i < numberOfSensors; i++) {
        if (reportData[i].xSemaphore != NULL) {
            xSemaphoreTake(reportData[i].xSemaphore, portMAX_DELAY);
            float f = reportData[i].filtered;
            xSemaphoreGive(reportData[i].xSemaphore);

            dtostrf(f, 6, 2, fFil);
            safePrintf(">%s:%s\n", reportData[i].sensorName, fFil);
        }
    }

```

```
        safePrintf("
            ----------------------------------------------------------------------------------------------------\n"
            );

        vTaskDelay(pdMS_TO_TICKS(delay));
    }
}
```

Listing 3: ReportTask.cpp